

Multi-Label Ambiguity Detection in Software Requirements using Language Models

Paniz Oghabi

Data Science, Analytics, and Artificial Intelligence
Carleton University, Canada
 panizoghabi@cmail.carleton.ca

Nafiseh Kahani

Systems and Computer Engineering
Carleton University, Canada
 nafiseh.kahani@carleton.ca

Abstract—In software engineering, requirements specification plays an important role in capturing and analyzing stakeholders' needs. However, as software requirements are typically written in natural language, they usually have ambiguity. Identifying and understanding the ambiguity in requirements enables development teams to prevent misunderstandings, address defects earlier, and reduce project costs and failures. While most recent studies focus on detecting only one or two types of ambiguity, this study investigates whether language models (LMs) can detect multiple types of ambiguity, specifically structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness. We collected 1,266 real-world requirements from the public requirements (PURE) dataset, which includes software requirements specifications from several domains, and labeled them using established methods. Two LMs, DistilBERT and RoBERTa, were fine-tuned using the labeled dataset (split into training and validation sets). Model performance was evaluated on an independent test set of 255 manually annotated requirements. RoBERTa achieved a higher macro-averaged F1-score (75.61%) compared to DistilBERT (73.15%). Applying back-translation (BT) data augmentation to address class imbalance further improved RoBERTa's macro-averaged F1-score to 78.84%. Compared to existing techniques, our multi-label method achieved approximately 12% and 7% higher F1-scores in scope and structural ambiguity detection, respectively, while demonstrating slightly lower results for anaphoric ambiguity (about 1.8%) and vagueness detection (about 3%).

Index Terms—Ambiguity Detection, Software Requirements Specifications, Language Models (LMs), Defects, Natural Language Processing (NLP), Multi-label Classification

I. INTRODUCTION

Software requirements determine what a system or an application requires to meet stakeholders' needs. Therefore, it is crucial that they are expressed in a precise and clear way. Requirements are mostly written in natural language [1], which allows for flexibility and widespread understanding [2]. Several studies have identified different classifications of defects in software requirements specifications, such as ambiguity, incorrectness, inconsistency, omission, superfluity, and non-conformance to standards [3]. Among these, ambiguity in software requirements is identified as one of the major factors in software quality [4]. Ambiguity occurs when a requirement statement can be interpreted in multiple ways, contributing to confusion, errors, and increased development costs and time [5]. During software system development, unresolved ambiguity can lead to disagreements between stakeholders and developers, which can potentially cause software failures

[6]. A notable real-world example is NASA's Mars mission failures in 1998–1999, where ambiguous interface specifications between software systems contributed to catastrophic losses worth hundreds of millions of dollars [7]. Such cases highlight the critical importance of addressing ambiguities early in the development process, as the cost of corrections and rework increases significantly in later stages of a project [8]. Therefore, precise and unambiguous requirements are essential for software quality and successful system development.

Traditional ambiguity detection methods, such as manual reviews, structured walkthroughs, scenario-based reading [9], and checklists [10], depend on human expertise, limiting their effectiveness for large projects [11], [12]. Current methods for detecting ambiguity in software requirements rely on rule-based methods [13], and machine learning (ML) models [14]. However, these methods have limitations, such as dependency on predefined rules or dictionaries, a narrow focus on one or two types of ambiguity, or just detecting whether it has ambiguity or not [12], [15]. Therefore, there is a critical need for an automated approach capable of detecting different types of ambiguity.

In this research, we propose a hybrid approach integrating established rule-based and ML methods to label four types of ambiguity, consisting of structural ambiguity, anaphoric ambiguity, scope ambiguity, and vagueness. We employ established methods: Score-Based Ambiguity Detector and Resolver (SBADR) [16] for structural ambiguity, TAPHSIR [14] for anaphoric ambiguity, rule-based heuristic [10], [17] for scope ambiguity, and predefined word lists [13], [18] for vagueness detection. Our labeled dataset, generated from these methods, is then used to train two language models (LMs), RoBERTa [19] and DistilBERT [20], for automatic detection of ambiguity types. Evaluating the trained models' performance on a test set dataset of 255 unseen requirements annotated by human experts showed that RoBERTa achieved higher macro-averaged metrics, including a recall of 88.24%, precision of 67.78%, F1-score of 75.61%, and hamming accuracy of 83.17%. After fine-tuning both models with back-translation (BT) augmented data, RoBERTa maintained superior performance across all metrics, outperforming DistilBERT by 5.03% in macro-averaged F1-score. Although our approach suppressed existing methods on scope and structural ambiguity detection (achieving F1-scores of 79.52% and

74.22%, respectively), it showed slightly lower performance for anaphoric ambiguity (89.80% vs. 91.67%) and vagueness detection (71.82% vs. 74.77%).

The main contributions of this work include:

- A unified approach for detecting multiple ambiguity types, eliminating the need for separate detection systems.
- Fine-tuned RoBERTa and DistilBERT (pre-trained on corpora of different sizes) on our requirements dataset labeled by existing methods and comprehensively evaluated their ability to detect ambiguity types, both with and without BT augmentation.
- Demonstration of performance improvements over existing ambiguity detection methods while maintaining competitive results across all ambiguity types on a human-annotated test set of 255 unseen requirements.

The remainder of this paper is organized as follows: section II explains ambiguity types and provides the rationale for focusing on specific types. Section III provides a literature review of existing ambiguity detection methodologies. Section IV describes our research methodology, including data creation and model development. Section V presents experimental results and analyses. Section VI discusses potential threats to validity, and Section VII summarizes our research conclusions and recommendations for future work.

II. BACKGROUND

A. Types of Ambiguity

Ambiguity generally occurs when a sentence or a phrase has more than one meaning or interpretation [21]. Riaz et al. [22] categorize ambiguity into the following types:

- Lexical ambiguity: occurs when a single word can be interpreted in different ways [23]. This type divides into *homonymic*, where one word has several unrelated meanings (e.g., “bank” meaning “river bank” and “financial institution”) and *polysemic*, where a word has related meanings sharing a common origin (e.g., “head” as a body part versus a leader of an organization).
- Structural (syntactical) ambiguity: arises when a sentence has different grammatical structures, and each interpretation has a different meaning. It includes subtypes, such as *attachment* ambiguity and *coordination* ambiguity. Attachment ambiguity occurs when phrases can attach to different parts of the sentence. For example, in the requirement “The TCS shall have the functionality to process UAV mission plans from other TCSs” it is unclear whether the prepositional phrase “from other TCSs” modifies the verb “process” or the noun phrase “UAV mission plans”. Coordination ambiguity involves multiple conjunctions or one conjunction with a modifier. “All telescope, instrument, and detector control information is to be available at any operation level” is considered a coordinator ambiguity. The first interpretation can be “control information” modified to “telescope”, “instrument”, and “detector”, or just “detector”. In addition, Osama et al. [16] identify another subtype *analytical ambiguity*,

caused by unclear roles of sentence constituents. For the requirement, “The user shall be able to select pretty-printed text files to display”, two parses are possible: (1) “pretty-printed” modifies “text files” (select files that have already been formatted); (2) “pretty-printed text” forms a compound noun, and the user selects the files containing such text.

- Semantic ambiguity: is present when a grammatically correct sentence can be interpreted in multiple ways because the context is confusing or unclear. Two common subtypes are *scope* ambiguity and *anaphoric* ambiguity. Scope ambiguity occurs when a sentence has more than one logical interpretation in the context [10]; for example, “Each transaction must be approved by a manager” means (1) one designated manager approves all transactions or (2) every transaction requires approval, but different managers may approve different transactions. Anaphoric ambiguity appears when a pronoun can refer to multiple antecedents, as in “The system processes the data and stores it in the database” where “it” could denote either the system or the data.
- Pragmatic ambiguity: occurs when sentence interpretation depends on contextual factors beyond the text itself [24]. This requires both linguistic elements (surrounding textual context) and extra-linguistic factors (situational circumstances, domain knowledge, and communicative expectations of stakeholders) [25], [26]. Ferrari and Gnesi [26] provide several detailed examples highlighting pragmatic ambiguity in software requirements.

In addition, Berry et al. [10] further recognize vagueness and language errors as additional sources of interpretational problems in requirements specifications. These categories, while distinct from classical ambiguity types, present similar challenges in achieving precise and unambiguous communication between stakeholders:

- Vagueness: occurs when a requirement lacks precise measurability or admits borderline cases (e.g., “The system shall provide a fast response time”, where “fast” is vague because the acceptable threshold is undefined).
- Language errors: arise when incorrect grammar, punctuation, and word choice lead to unintended interpretations. For example, in “Users must update his password every 90 days”, the pronoun–antecedent mismatch (plural users vs. singular his) makes it unclear whether the rule applies to each individual user or to a single shared account. However, these errors often manifest as other types of ambiguity and lack specific methods for detection.

Table I summarizes the main ambiguity categories identified in requirements engineering literature. However, not all types are equally suitable for automated detection given current technological capabilities and practical constraints.

B. Rationale for Exclusion of Certain Ambiguity Types

We exclude certain types of ambiguity, such as lexical ambiguity, pragmatic ambiguity, and language errors, from

TABLE I
TYPES OF AMBIGUITY [10], [22]

Ambiguity Types	Definition
Lexical Ambiguity	A word can have multiple meanings.
Structural Ambiguity	A sentence can be parsed in more than one grammatical structure, each causing a different meaning.
Semantic Ambiguity	A grammatically correct sentence that conveys multiple interpretations.
Pragmatic Ambiguity	Interpretation depends on external context or domain knowledge.
Vagueness	Specifications are so imprecise that they cannot be measured clearly.
Language Errors	Errors arise from grammar mistakes, incorrect punctuation, or inappropriate word choice.

our detection approach due to the lack of reliable detection methods or the necessity of additional contextual information beyond our work. In lexical ambiguity, though the goal is to find words with multiple meanings, methods using lexical databases like WordNet or BabelNet tend to over-identify ambiguity [27]. Since many words naturally have several dictionary meanings, almost every requirement would include at least one such word.

Pragmatic ambiguity is also removed because it requires context beyond the requirement statement itself. Although Riaz et al. [22] mention referential ambiguity as a common subset of the pragmatic category, it overlaps with anaphoric ambiguity [10], and also, based on existing pragmatic detection methods, requires extensive background documentation or project-specific knowledge bases [24], [25]. Since our analysis is limited to individual requirement statements, pragmatic ambiguity detection remains beyond our scope. We also exclude language errors due to the absence of systematic detection methods, as these typically manifest through other ambiguity categories.

C. Scope of Ambiguity Detection

Based on the rationale above, this work targets four key ambiguity types selected for their prevalence and suitability for automated detection. We address the two subtypes of semantic ambiguity, scope ambiguity and anaphoric ambiguity, as separate categories due to the distinct detection methods applied to each. Therefore, our work targets four ambiguity types: *structural ambiguity*, *scope ambiguity*, *vagueness*, and *anaphoric ambiguity*. These categories can be reliably detected using available methods and cover the most important ambiguity problems in requirements.

III. RELATED WORK

Automated ambiguity detection in software requirements has gained significant attention as organizations seek to improve requirement clarity while reducing manual effort. Various methods have been proposed to detect ambiguity, with some focusing on binary classification (ambiguous vs. unambiguous) while others concentrate on one or two ambiguity

types, or a few more, using fixed dictionaries. This section reviews key existing contributions in ambiguity detection, examining their methodologies and limitations.

Zait and Zarour [27] propose a technique for detecting and resolving lexical ambiguity and semantic ambiguity (specifically, scope ambiguity) in raw natural language requirements using Natural Language Processing (NLP) and semantic web techniques. They employ POS tagging and BabelNet for detection, and use n-gram context analysis combined with a graph-based centrality algorithm for lexical disambiguation. For semantic ambiguity, they identify based on a list called “blacklist”, and they do not provide a clear explanation for semantic ambiguity resolution. However, their evaluation is limited to a single case study (RALIC project), and the technique focuses primarily on lexical ambiguity and semantic ambiguity while not addressing other ambiguity types.

Ezzini et al. [28] utilize different methods, including SpanBERT-based, traditional ML, and NLP-based techniques for detecting and resolving anaphoric ambiguity. They find supervised ML effective in ambiguity detection, while SpanBERT excels in resolution. Additionally, Ezzini et al. [14] develop an automated system that combines feature-based ML for anaphoric ambiguity detection with a fine-tuned SpanBERT model for anaphora resolution.

Osama et al. [16] present SBADR, an automatic technique for detecting and resolving structural ambiguity in software requirements. SBADR generates k-best parse trees for each sentence with Stanford CoreNLP and then applies four filtering pipelines to eliminate meaningless interpretations. If more than one tree remains, the tool flags the sentence as ambiguous and shows the bracketed alternatives to the analyst. The tool achieved 99% recall and 77% F1-score on 126 requirements and reached 100% recall on coordination ambiguity and 90% on attachment ambiguity in complex scenarios. However, the technique is limited to just syntactic ambiguity detection and relies heavily on the underlying Stanford CoreNLP parser’s capabilities.

Lami et al. [13] develop Quality Analyzer for Requirement Specifications (QuARS), an automatic tool for detecting various linguistic defects in requirements, including vagueness and weakness. The tool aims to achieve software quality by performing a systematic analysis of requirements specifications to identify defects that could contribute to problems in the development phases. Tjong et al. [18] present Systemized Requirements Engineering Environment (SREE), a prototype ambiguity finder that has 100% recall but a lower precision. SREE takes a lexical analysis approach through 10 different categories, such as vague terms and coordination ambiguity. Both SREE and QuARS tools rely on predefined keyword lists that may produce many false positives.

Subedi et al. [15] investigate transfer-learning with data augmentation for binary ambiguity detection. They pre-train a ULMFiT language model on 1,460 software requirement sentences from 16 industrial and academic projects, then fine-tune a classifier on a manually labelled subset of 483 requirements (241 ambiguous, 242 unambiguous). They increase the la-

belled set by 25% via Spanish BT augmentation, as it gives the highest mean Bilingual Evaluation Understudy (BLEU) score of 43.9% among ten languages. In 10-fold cross-validation, the augmented ULMFiT model surpassed classical baselines, including SVM, Logistic Regression, and multinomial Naive Bayes classifiers, demonstrating that BT can measurably boost performance when training data is limited.

Izhar et al. [12] propose a hybrid framework combining rule-based techniques with ML models for ambiguity detection. Their method first applies expert-defined rules to flag ambiguous phrases, then uses TF-IDF features with ML classifiers to improve detection performance. They found that sentiment and ambiguity have a weak relationship ($r = 0.22$), suggesting that sentiment analysis may not be effective for ambiguity detection. They evaluated their approach on a dataset of 1,553 requirements across 17 project domains (with 1,371 non-ambiguous and only 185 ambiguous requirements), achieved 97% accuracy and 92% F1-score with Random Forest, significantly outperforming traditional rule-based tools and other ML models. However, the complete list of ambiguous terms and rules is not publicly shared, and the dataset remains imbalanced.

Bashir et al. [29] demonstrated an industrial study exploring the use of large language models (LLMs) for requirements ambiguity detection and providing explanations to support addressing ambiguities in safety-critical domains such as the railway industry. Using different shot configurations, they evaluated three LLMs, such as Phi-3 Mini, Llama-3, and Qwen-2.5. Results showed that performance generally improved with more demonstrations, with 10-shot prompting yielding statistically significant gains compared to 0-shot and 1-shot. While this work provides valuable insights into large language model capabilities for requirements analysis in industrial settings and includes explanation generation, its binary classification approach does not identify the specific types of ambiguity, and the datasets used are not publicly available.

Despite advancements, existing methods and tools still have limitations. Some handle only one or two ambiguity classes [16], [27], [28], while others attempt broader coverage but rely on static word lists [18]. Another issue is the lack of transparency in labeling methodologies, as many studies do not provide clear details about how they annotate data for ambiguity detection [12]. Most methods also reduce the task to a binary decision (“ambiguous” vs. “unambiguous”) without identifying the specific ambiguity types present [15]. To bridge these gaps, our study introduces a transformer-based, multi-label classification approach that detects structural, scope, anaphoric, and vagueness ambiguities. By combining systematic labeling methodologies, data augmentation, and transformer models, we aim to improve coverage and applicability in requirements engineering.

IV. METHODOLOGY

The goal of this study is to determine whether LMs can detect different types of ambiguity in software requirements. Figure 1 illustrates the overall pipeline followed in this study,

divided into two major phases: data creation and model development. In the first phase, (1) we collected requirements from real-world software projects, (2) preprocessed the data, and (3) labeled them for different ambiguity types using established approaches from previous studies. In the model development phase, (4) we split the labeled dataset into training and validation sets, and (5) applied data augmentation to address class imbalance within the structural ambiguity, scope ambiguity, and anaphoric ambiguity. Finally, (6) we chose two transformer-based models (DistilBERT [20] and RoBERTa [19]) to (7) fine-tune with two setups: baseline models, which were trained on only the training set, and augmented models, which were trained on both the training set and augmented data to provide a superior multi-label classification model for ambiguity detection.

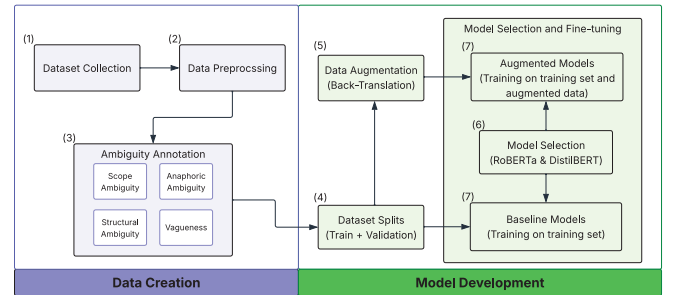


Fig. 1. Overall Methodology for Detecting Ambiguity Types in Software Requirements.

A. Dataset Creation

We collected software requirements from real-world projects in the public requirements (PURE) dataset [30], performed preprocessing to clean the data, and then created labels for four ambiguity types: structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness. Note that these types of ambiguity labels were not present in the dataset; we independently generated them using established methods. This approach ensures no pre-existing labels are used, thus preventing potential information leakage to the LMs.

1) *Data Collection*: We utilized the PURE dataset, which includes 79 real-world requirements documents from industrial and academic projects [30]. As the projects were stored in different formats, such as PDF, DOC, HTML, and XML, and some documentation was presented in tabular format, extracting the requirements from each file was challenging. To address this issue, we used the text format version of the PURE dataset provided by Luitel et al. [31], which contains 40 projects that were extracted primarily through manual cleanup. From this set, we selected seven documents in distinct domains, which contained 1,266 requirements across various areas, including weather, control systems, administration, astronomy, communication, visualization, and computer games. This diversity in project types and structures was essential for producing more generalizable results. The requirements from these projects required additional manual verification, as

they included information beyond the requirements, and some of the requirements did not exactly match those documented in the original PURE dataset. Several requirements had been incorrectly merged across different lines, resulting in inaccurate documentation. For instance, in the Clarus Low project, the original requirement “F-214D1 CS shall accept surface condition data derived from surface image,” was inaccurately documented by Luitel et al. [31] as “F-214D1 CS shall accept surface condition “covered” or “flooded””.

2) *Data Preprocessing*: The preprocessing steps included removing requirement IDs, HTML, email, and other tags from the text. Numbers were replaced with the token <NUM> using regular expressions. It should be noted that stop words were retained, as they can potentially contribute to various types of ambiguity that the model aims to detect. Following the data collection and preprocessing, we proceeded to label the requirements based on their ambiguity types.

3) *Ambiguity Annotation*: In this section, we explain our approach to creating a multi-label dataset. We used existing tools, rule-based techniques, and newly combined methods to generate labels that capture different types of ambiguity.

Anaphoric ambiguity. We used the TAPHSIR tool [14] to detect anaphoric ambiguity. This method uses two ensemble classifiers, trained on linguistic features and contextual embeddings, respectively, to classify triples of context, pronoun, and candidate antecedent as “correct”, “incorrect”, or “inconclusive”. The tool then applies rules based on prediction probabilities to determine whether each pronoun occurrence is anaphorically ambiguous.

Scope ambiguity. Zait and Zarour [27] detect scope ambiguity when a sentence contains a quantifier from their defined blacklist: any, few, little, many, much, several, and some. However, detecting ambiguity based solely on the presence of a quantifier is insufficient and may produce many false positives. For example, the requirement “The system shall log some user activities” does not exhibit scope ambiguity, as it contains only one quantifier without interacting elements. Furthermore, their blacklist omits universal quantifiers, such as all, which can also trigger scope ambiguity [10]. To establish a more precise labeling process, we adopted a rule-based heuristic identified by Kamath et al. [17]. According to their heuristic, a requirement is flagged as scope-ambiguous if it matches at least one of these patterns:

- Quantifier–Quantifier: Contains two or more quantifiers.
- Quantifier–Negation: Contains at least one quantifier and one negation.
- Quantifier–Adverb: Contains at least one quantifier and one frequency adverb.

We implemented these rules in a Python script that tokenizes each sentence, detects keywords, and returns “True” when any condition is met. Our implementation was validated using the dataset provided by Kamath et al. [17], confirming accurate identification of their intended scope-ambiguous sentences. This successful validation ensures the reliability of applying this refined heuristic-based method to label our software requirements.

Vagueness. We used a combined list of 466 vague terms derived from QuARS [13] and an additional list provided by SREE [18]. Both sources offer lists of terms commonly associated with vagueness. Previous research (e.g., [32]) suggested expanding these lists with a quantifier like “some” for broader vagueness detection. However, since quantifiers often relate to scope ambiguity rather than vagueness, and including them might overlap with our existing scope ambiguity detection method, we refrained from adopting this extension.

Other studies, such as [27], [33], identify adjectives and adverbs as the most potentially vague. Although this method definitely leads to 100% recall, it results in numerous false positives since not all adjectives and adverbs inherently cause vagueness. Despite their comprehensive nature, predefined term lists might not capture all domain-specific vagueness, highlighting an opportunity for future improvement. Moreover, even advanced domain-aware methods integrating Part-of-Speech (POS) tagging and specialized domain dictionaries, such as the method proposed by Zakeri-Nasrabadi and Parsa [34], achieved only modest performance (F1-score of 40%) with lower recall for vague adjectives and adverbs. Given these limitations, we ultimately relied on the combined list of vague terms provided by QuARS and SREE to label the requirements with a vagueness label.

Structural ambiguity. For this type of ambiguity, we used SBADR [16] that detects structural ambiguity by generating multiple parse trees via Stanford CoreNLP for each sentence. The paper proposes four filtering pipelines to choose the most suitable trees. If more than one valid interpretation remains after filtering, the sentence is marked as structurally ambiguous. By using this method, we can identify potential structural ambiguity in different domains. As the original implementation is not publicly available, we developed our own implementation based on the exact description in the source paper. To ensure our implementation was faithful to the original work, we validated it using the examples provided in the SBADR paper and confirmed that it replicated the expected outcomes.

By applying these methods, we created a dataset with five columns: *requirement text*, *structural ambiguity*, *scope ambiguity*, *anaphoric ambiguity*, and *vagueness*. Out of the 1,266 requirements analyzed, 939 were identified with structural ambiguity, 126 with anaphoric ambiguity, 221 with scope ambiguity, and 551 with vagueness. Note that a requirement may contain multiple ambiguity types.

B. Model Development

In the Model Development phase, we built multi-label classification models to detect multiple types of ambiguity in software requirements. First, we split the labeled dataset into training and validation sets. To address class imbalance, we applied data augmentation using a BT method, targeting the minority class within each of the structural ambiguity, scope ambiguity, and anaphoric ambiguity. We fine-tuned two transformer-based LMs, DistilBERT and RoBERTa, in two separate experimental settings: (1) training on the original

dataset only, and (2) training on the original dataset plus augmented minority class data. The detailed evaluation and performance comparison of these models on an independently labeled test set are detailed in the subsequent evaluation section (Section V).

1) *Dataset Splits*: The dataset was split into a training set and a validation set. This allocated 80% (1,012 samples) for training and 20% (254 samples) for validation. We used the iterative stratification method [35], which maintains the distribution of label combinations across both sets. The training set was used for fine-tuning LMs, while the validation set was used to track model performance and prevent overfitting during training.

2) *Data Augmentation*: Since the training set was imbalanced, we employed BT to mitigate this issue. BT preserves semantic content and underlying structural patterns while making controlled lexical variations, making it highly suitable for ambiguity classification tasks. Following the methodology explained by Subedi et al. [15], we selected Spanish as the target language, as it yields higher BLEU scores [36] compared to ten other languages commonly used in requirements documentation. The BT process (English $\rightarrow$ Spanish $\rightarrow$ English) was implemented using the multilingual translation model 'facebook/nllb-200-distilled-600M' [37], which belongs to the NLLB-200 family and is designed to provide high-quality translations across 200 languages. We augmented requirements that lacked structural ambiguity or contained either anaphoric ambiguity or scope ambiguity. This targeted method resulted in the following improvements: (i) For structural ambiguity, non-ambiguous requirements increased from 261 to 516, while structurally ambiguous requirements grew moderately from 751 to 966. (ii) Anaphoric ambiguity doubled from 101 to 201, and (iii) scope ambiguity similarly increased from 173 to 345. (iv) Vagueness showed balanced growth, with ambiguous requirements increasing from 459 to 677. These numbers demonstrate that the rare types approximately doubled in size, reducing class imbalance. After augmentation, the dataset contained 1,482 samples. Although some imbalance persists as requirements may have several types of ambiguity at the same time, and the original gaps were large, achieving full balance would require far more augmentation or manual annotation.

We measured BLEU [36] and Metric for Evaluation of Translation with Explicit Ordering (METEOR) [38] scores using the Python library "evaluate"¹. The BLEU score measures translation quality based on the precision of overlapping word sequences (n-grams) between the back-translated text and the original, whereas the METEOR score combines precision, recall, and synonym matching to evaluate semantic similarity. The augmented dataset achieved an average BLEU score of 0.422 and an average METEOR score of 0.772. A BLEU score above the 40%-50% threshold indicates high translation quality, suggesting that labels for the augmented requirements remain reliable [15], [39]. Similarly, the METEOR score of

0.772 demonstrates strong semantic preservation and word alignment quality in the back-translated requirements [38].

3) *Model Selection and Fine-tuning*: In selecting LMs, we prioritized freely available models suitable for fine-tuning, accordingly excluding proprietary models such as GPT-4 [40]. Moreover, model size is another criterion that needs to be considered; we avoided large-scale models like Llama [41] due to their extensive GPU requirements for fine-tuning. Alharbi [42] utilized DistilBERT [20] for ambiguity detection, and this model aligns well with our selection criteria. Consequently, we selected DistilBERT as our first model; it is a compact version of BERT [43], retaining approximately 97% of BERT's performance while being 40% smaller and 60% faster. Our second model, RoBERTa [19], was chosen for its pre-training on a significantly larger and more diverse corpus compared to BERT. By including RoBERTa, we explored whether models pre-trained on broader datasets yield improved ambiguity detection performance.

Our implementation utilized DistilBertTokenizer and DistilBertForSequenceClassification for DistilBERT, and RobertaTokenizer and RobertaForSequenceClassification for Roberta, all from the Transformers library [44]. These tokenizers convert the preprocessed requirement texts into the numerical input formats expected by each model, handling subword tokenization and the addition of special tokens. Both models were fine-tuned under two experimental setups: first, training exclusively on the training set (baseline models), and second, training on an expanded dataset combining the training set with additional augmented samples generated via BT to address class imbalance.

V. EVALUATION

We validate our approach by creating a manually annotated test set, defining our research questions, and presenting experimental results that address these questions.

A. Manual Test Set

The annotated test set contained 255 requirements from the PURE dataset, which were not included in the training or validation sets. To reduce bias, the requirements were manually labeled by three annotators: two experts in requirements engineering (including one of the authors) and one linguistics expert. Each annotator was assigned binary labels (0 or 1) across the four ambiguity types, where 0 indicates an unambiguous requirement and 1 indicates ambiguity.

In order to obtain final labels, we employed majority voting. For each ambiguity type per requirement, if at least two annotators assigned the same label, it was adopted as the definitive label. We calculated Fleiss' kappa [45] for each ambiguity type to measure inter-annotator agreement and reported the average agreement across all types. According to Landis and Koch [46], a kappa value between 0.41–0.60 indicates moderate agreement, while a value between 0.61–0.80 represents substantial agreement. Based on Table II, structural ambiguity, anaphoric ambiguity, and scope ambiguity show substantial agreement, while vagueness has moderate agreement levels.

¹<https://github.com/huggingface/evaluate>

Since the average kappa score lies within the substantial agreement range (0.61–0.80), we consider the annotations sufficiently reliable for evaluating our approach.

After annotation, we applied the preprocessing pipeline described in Section IV-A2 (e.g., tag removal, number masking) to the 255 requirements so the test data matched the format used during training.

TABLE II
FLEISS' κ COEFFICIENT FOR EACH TYPE OF AMBIGUITY.

Ambiguity Types	Fleiss' κ coefficient
Structural Ambiguity	0.6131
Scope Ambiguity	0.6147
Anaphoric Ambiguity	0.7255
Vagueness	0.5077
Overall	0.6153

B. Research Question

The evaluation phase addresses the following research questions:

- **RQ1 Which fine-tuned language model achieves the highest baseline performance for multi-label ambiguity detection?** This research question aims to identify the most effective fine-tuned transformer-based language model for detecting multiple types of ambiguity. We evaluate the performance of RoBERTa and DistilBERT models on the annotated test set.
- **RQ2 Which language model provides the best performance after being fine-tuned with BT augmentation?** By fine-tuning both RoBERTa and DistilBERT on the expanded dataset (training set combined with augmented data), this research question assesses the effectiveness of our data augmentation strategy. We aim to identify the definitive best-performing model and assess whether the best-performing baseline model identified in RQ1 retains its superior performance after applying data augmentation.
- **RQ3 How does our unified multi-label model, the best-performing model from RQ2, compare to previous detection methods?** The primary objective of this research question is to determine whether a unified multi-label classification approach performs comparably or better than traditional ambiguity detection methods. We conduct comparative analyses across each ambiguity type to assess the advantages and potential trade-offs of using a single unified model.

C. Evaluation Metrics

To evaluate our multi-label classification model for ambiguity type detection, we use two distinct categories of metrics: *Individual Ambiguity Type Performance* and *Multi-Label Performance*.

1) *Individual Ambiguity Type Performance*: We compare our method with existing ambiguity detection methods using accuracy, precision, recall, and F1-score, which are commonly used in similar studies [12], [42]. All metrics range from 0 to 1, with higher values indicating better performance.

2) *Multi-Label Performance*: We compute macro-averaged precision, recall, and F1-score across all ambiguity types to evaluate the overall performance of multi-label classification models [47], [48]. For example, the recall of the multi-label classification model is calculated as the macro average of individual recall scores from each ambiguity type. Macro averaging weights every label equally, ensuring that rare ambiguity types contribute equally to the overall evaluation as more frequent ones.

We do not report standard accuracy for the performance of our multi-label classification models because it requires all labels per instance to be correct, which fails to distinguish between false positive and false negative errors and can mask performance issues with rare labels [49]. Instead, we use hamming accuracy, which measures the proportion of correctly predicted labels across all samples and is widely adopted for multi-label evaluation [50].

D. Results and Analysis

RQ1. Which fine-tuned language model achieves the highest baseline performance for multi-label ambiguity detection? We fine-tuned DistilBERT and RoBERTa on the training set and evaluated each model on the annotated test set separately. Table III shows the macro-averaged metrics and hamming accuracy of the two models. Because macro-averaged F1-score combines precision and recall while weighting each ambiguity label equally, we consider it as the main selection metric. RoBERTa attained the higher macro-averaged F1-score (75.61%), outperforming DistilBERT by 2.5%. This indicates that RoBERTa offers the best trade-off between finding true ambiguous cases (high recall) and avoiding false positives (adequate precision). Also, the hamming accuracy of RoBERTa is about 2% more than DistilBERT. This superior performance can be attributed to key advantages in architecture and pre-training of RoBERTa. It has a larger model size with 125 million parameters compared to DistilBERT's 66 million parameters and was pre-trained on a more extensive and diverse corpus (approximately 160GB of text) than DistilBERT. These factors likely provide more robust feature representations for the complex ambiguity detection task.

TABLE III
COMPARISON OF DISTILBERT AND ROBERTA IN DETECTING AMBIGUITY TYPES.

Metrics (%)	DistilBERT	RoBERTa
Precision	65.33	67.68
Recall	85.14	88.24
F1-score	73.15	75.61
Hamming Accuracy	81.30	83.17

Precision, Recall, and F1-score are macro-averaged.

RQ2. Which language model provides the best performance after being fine-tuned with BT augmentation? To evaluate the impact of data augmentation, we fine-tuned DistilBERT and RoBERTa using an expanded training set, incorporating additional back-translated samples targeting minority ambiguity classes (structural ambiguity, scope ambiguity, and anaphoric ambiguity), as described in Section IV-B2. The results demonstrated that RoBERTa remained the superior model. After augmentation, RoBERTa achieved a macro-averaged F1-score of 78.84%, an improvement of 5.03% over DistilBERT’s score of 73.81%. RoBERTa also outperformed DistilBERT across all other metrics (Table IV). Compared to the baseline results in RQ1, RoBERTa’s macro-averaged F1-score improved from 75.61% to 78.84%, confirming that the BT augmentation strategy effectively enhanced model performance. This analysis establishes the combination of RoBERTa and BT augmentation as the optimal pipeline for the ambiguity detection task addressed in this study.

TABLE IV
COMPARISON OF DISTILBERT AND ROBERTA IN DETECTING AMBIGUITY TYPES AFTER BACK-TRANSLATION AUGMENTATION.

Metrics (%)	DistilBERT	RoBERTa
Precision	66.27	69.79
Recall	85.08	90.69
F1-score	73.81	78.84
Hamming Accuracy	82.68	85.24

Precision, Recall, and F1-score are macro-averaged.

RQ3. How does our unified multi-label model, the best-performing model from RQ2, compare to previous detection methods? We compared the performance of our unified multi-label model (the BT-augmented best model from RQ2) against four existing methods, each specialized for detecting a single ambiguity type: SBADR for structural ambiguity, rule-based semantic operators for scope ambiguity, TAPHSIR for anaphoric ambiguity, and predefined word lists for vagueness. The comparative results are presented in Table V. We selected the F1-score as the primary evaluation metric because it provides a balanced assessment of precision and recall.

For structural ambiguity, our multi-label model significantly outperformed SBADR, achieving an F1-score of 74.22%, compared to SBADR’s 61.86%. This improvement was consistently observed across all evaluation metrics, including accuracy, precision, and recall. Similarly, for scope ambiguity, our model attained an F1-score of 79.52%, surpassing the rule-based method’s score of 72.53%. Moreover, accuracy improved by 3.15%, showing a clear advantage in understanding this type of ambiguity.

In contrast, for anaphoric ambiguity, the existing tool, TAPHSIR, achieved a slightly higher F1-score of 91.67% compared to our model’s 89.80%. Although both models achieved perfect recall (100%), TAPHSIR’s superior precision indicates better suitability for this specific task; however, the performance difference remains minor. A similar trend was observed for vagueness, where the predefined word-list

TABLE V
PERFORMANCE COMPARISON BETWEEN PROPOSED AND EXISTING METHODS FOR EACH AMBIGUITY TYPE

Ambiguity Types	Metrics (%)	Existing Works	Model
Structural Ambiguity	Accuracy	56.30	74.02
	Precision	48.91	63.76
	Recall	84.11	88.79
	F1-score	61.86	74.22
Scope Ambiguity	Accuracy	90.16	93.31
	Precision	60.00	70.21
	Recall	91.67	91.67
	F1-score	72.53	79.52
Anaphoric Ambiguity	Accuracy	98.43	98.03
	Precision	84.62	81.48
	Recall	100	100
	F1-score	91.67	89.80
Vagueness	Accuracy	78.74	75.59
	Precision	67.80	63.71
	Recall	83.33	82.29
	F1-score	74.77	71.82

The existing methods’ results for each ambiguity type are from different, specialized studies.

technique attained an F1-score of 74.77% compared to our model’s 71.82%.

While our unified multi-label model provided a highly effective technique for structural and scope ambiguities, its performance was only narrowly lower for anaphoric ambiguity and vagueness, with the existing specialized methods resulting in a small margin of approximately 2% and 3% in F1-score, respectively. This trade-off underscores the practical value of our unified approach. Prior research typically targets only one or two ambiguity types, necessitating the implementation of multiple disparate tools for comprehensive coverage. For instance, detecting both anaphoric ambiguity with TAPHSIR and structural ambiguity with SBADR requires managing methods built in different programming languages (e.g., Python and Java), along with their separate dependencies and output formats. Integrating these disparate outputs increases complexity and resource demands. Our unified model simplifies this process by identifying multiple ambiguity types within a single approach, providing a more practical and efficient solution. Given the effectiveness of LMs for ambiguity detection, we anticipate further performance improvements through prompt engineering and fine-tuning LLMs. To establish a baseline for comparison, we tested Llama-3 [51] and Gemma 2 [52] using a basic prompt (Figure 2) that only provides an explanation of ambiguity and one example for each type. These models achieved modest performance, with macro-averaged F1-scores of 19.96% and 32.60%, respectively, on our test set.

VI. THREATS TO VALIDITY

Our study faces three main threats to its validity. While our final model achieved a high macro-averaged recall of 90.69%, macro-averaged precision was comparatively lower at 69.79%. This difference between high recall and lower precision is

Task Description

You are an expert in software requirements analysis tasked with analyzing requirements for different types of ambiguity.

Inputs

Ambiguity Types: {ambiguity_descriptions} that contain definitions and examples of four ambiguity types: structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness.

Requirement to Analyze: "{requirement}".

Task Instructions

- 1) Analyze the provided requirement for each of the four ambiguity types.
- 2) For each type, return 1 if present, 0 if absent.
- 3) Note that each requirement may have multiple ambiguity types.
- 4) Return results in JSON format following the structure shown in Output Format below, without additional explanation.

Output Format

```
{"structural ambiguity": 0, "scope ambiguity": 1, "anaphoric ambiguity": 0, "vagueness": 1}
```

Fig. 2. Prompt for Ambiguity Analysis.

explained by the detailed results for each ambiguity type presented in Table V. The model's precision is notably lower for structural ambiguity (63.76%) and vagueness (63.71%), which brings down the macro-average. This is likely linked to the nature of the training data itself. The initial labeling methods employed, such as SBADR and predefined word lists, have limited precision, causing our model to inherit these labeling inaccuracies during fine-tuning. While the primary objective of this study was to assess the LMs' capability to detect ambiguity types given a dataset, a more precise examination of the training dataset is necessary to maintain its high quality for future applications. Moreover, persistent class imbalance is another limitation. Although BT augmentation mitigated this issue, it did not entirely resolve it. Ambiguity types like structural ambiguity and vagueness are naturally more frequent in real-world requirements, and this underlying distribution may still influence the model's learning process. Our work focuses solely on detecting ambiguity types and does not address their resolution. The potential interactions between different types during the resolution process are not examined in this work.

VII. CONCLUSION

This work presents an approach for detecting multiple ambiguity types in software requirements, including structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness, addressing limitations inherent in previous ambiguity detection methods. Our methodology included label-

ing requirements following established techniques, fine-tuning two transformer-based LMs (DistilBERT and RoBERTa), and employing BT augmentation to reduce class imbalance. We evaluated the models using a manually annotated test set of 255 requirements labeled by three experts. The annotations achieved an average Fleiss' kappa score of 0.62, indicating substantial annotator agreement. The fine-tuned RoBERTa model, enhanced with BT augmentation, achieved the highest performance with a macro-averaged F1-score of approximately 78.84%, surpassing baseline methods. This final model demonstrated considerable improvement over existing methods in detecting structural ambiguity (12% improvement) and scope ambiguity (7% improvement). Future research could focus on refining labels, particularly for vagueness and structural ambiguity, to improve model precision. Also, we plan to extend our methodology to include other ambiguity types, such as lexical ambiguity, pragmatic ambiguity, and language error, and explore domain-specific vagueness detection methods to provide wide coverage of ambiguities present in requirements specifications. Finally, integrating our ambiguity detection approach with automated issue-resolution systems represents another promising direction.

ACKNOWLEDGMENT

We would like to thank Pouyan Oghabi for his assistance in labeling the dataset. This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC).

REFERENCES

- [1] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, "Natural language processing for requirements engineering: A systematic mapping study," *ACM Computing Surveys (CSUR)*, vol. 54, no. 3, pp. 1–41, 2021.
- [2] A. Aurum and C. Wohlin, *Engineering and managing software requirements*. Springer, 2005, vol. 1.
- [3] A. A. Alshazly, A. M. Elfatraty, and M. S. Abougabal, "Detecting defects in software requirements specification," *Alexandria Engineering Journal*, vol. 53, no. 3, pp. 513–527, 2014.
- [4] A. Rani and G. Aggarwal, "Advanced practices to detect ambiguities and inconsistencies from software requirements," in *2018 International Conference on System Modeling & Advancement in Research Trends (SMART)*. IEEE, 2018, pp. 17–21.
- [5] D. M. Berry and E. Kamsties, "Ambiguity in requirements specification," in *Perspectives on software requirements*. Springer, 2004, pp. 7–44.
- [6] E. Kamsties, D. M. Berry, B. Paech, E. Kamsties, D. Berry, and B. Paech, "Detecting ambiguities in requirements documents using inspections," in *Proceedings of the first workshop on inspection in software engineering (WISE'01)*, vol. 13. Citeseer, 2001.
- [7] E. Euler, "The failures of the mars climate orbiter and mars polar lander—a perspective from the people involved," in *Guidance and control 2001*, 2001, pp. 635–655.
- [8] R. Van Megen and D. B. Meyerhoff, "Costs and benefits of early defect detection: experiences from developing client server and host applications," *Software Quality Journal*, vol. 4, pp. 247–256, 1995.
- [9] E. Kamsties and B. Peach, "Taming ambiguity in natural language requirements," in *Proceedings of the Thirteenth international conference on Software and Systems Engineering and Applications*, vol. 1315, 2000.
- [10] A. Handbook, "From contract drafting to software specification: Linguistic sources of ambiguity," 2003.
- [11] F. de Bruijn and H. L. Dekkers, "Ambiguity in natural language software requirements: A case study," in *Requirements Engineering: Foundation for Software Quality: 16th International Working Conference, REFSQ 2010, Essen, Germany, June 30–July 2, 2010. Proceedings 16*. Springer, 2010, pp. 233–247.

- [12] R. Izhar, S. N. Bhatti, and S. A. Alharthi, "Bridging precision and complexity: A novel machine learning approach for ambiguity detection in software requirements," *IEEE Access*, 2025.
- [13] G. Lami, S. Gnesi, F. Fabbrini, M. Fusani, and G. Trentanni, "An automatic tool for the analysis of natural language requirements," *Informe técnico, CNR Information Science and Technology Institute, Pisa, Italia, Setiembre*, 2004.
- [14] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Taphsir: towards anaphoric ambiguity detection and resolution in requirements," in *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 1677–1681.
- [15] I. M. Subedi, M. Singh, V. Ramasamy, and G. S. Walia, "Application of back-translation: a transfer learning approach to identify ambiguous software requirements," in *Proceedings of the 2021 ACM Southeast Conference*, 2021, pp. 130–137.
- [16] M. Osama, A. Zaki-Ismail, M. Abdelrazek, J. Grundy, and A. Ibrahim, "Score-based automatic detection and resolution of syntactic ambiguity in natural language requirements," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 651–661.
- [17] G. Kamath, S. Schuster, S. Vajjala, and S. Reddy, "Scope ambiguities in large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 738–754, 2024.
- [18] S. F. Tjong and D. M. Berry, "The design of sree—a prototype potential ambiguity finder for requirements specifications and lessons learned," in *International Working Conference on Requirements Engineering: Foundation for Software Quality*. Springer, 2013, pp. 80–95.
- [19] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [20] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [21] M. Luisa, F. Mariangela, and N. I. Pierluigi, "Market research for requirements analysis using linguistic tools," *Requirements Engineering*, vol. 9, pp. 40–56, 2004.
- [22] M. Q. Riaz, W. H. Butt, and S. Rehman, "Automatic detection of ambiguous software requirements: An insight," in *2019 5th International Conference on Information Management (ICIM)*. IEEE, 2019, pp. 1–6.
- [23] B. Gleich, O. Creighton, and L. Kof, "Ambiguity detection: Towards a tool explaining ambiguity sources," in *Requirements Engineering: Foundation for Software Quality: 16th International Working Conference, REFSQ 2010, Essen, Germany, June 30–July 2, 2010. Proceedings 16*. Springer, 2010, pp. 218–232.
- [24] A. Ferrari, G. Lipari, S. Gnesi, and G. O. Spagnolo, "Pragmatic ambiguity detection in natural language requirements," in *2014 IEEE 1st International Workshop on Artificial Intelligence for Requirements Engineering (AIRE)*. IEEE, 2014, pp. 1–8.
- [25] K. Aslam, F. Iqbal, A. Altaf, N. Hussain, M. G. Villar, E. S. Flores, I. D. L. T. Diez, and I. Ashraf, "Detecting pragmatic ambiguity in requirement specification using novel concept maximum matching approach based on graph network," *IEEE Access*, vol. 12, pp. 15 651–15 661, 2024.
- [26] A. Ferrari and S. Gnesi, "Using collective intelligence to detect pragmatic ambiguities," in *2012 20th IEEE International Requirements Engineering Conference (RE)*. IEEE, 2012, pp. 191–200.
- [27] F. Zait and N. Zarour, "Addressing lexical and semantic ambiguity in natural language requirements," in *2018 Fifth International Symposium on Innovation in Information and Communication Technology (ISIICT)*. IEEE, 2018, pp. 1–7.
- [28] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Automated handling of anaphoric ambiguity in requirements: a multi-solution study," in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 187–199.
- [29] S. Bashir, A. Ferrari, M. Abbas, P. E. Strandberg, Z. Haider, M. Saadatmand, and M. Bohlin, "Requirements ambiguity detection and explanation with llms: An industrial study," in *International Conference on Software Maintenance and Evolution*, 2025.
- [30] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "Pure: A dataset of public requirements documents," in *2017 IEEE 25th international requirements engineering conference (RE)*. IEEE, 2017, pp. 502–505.
- [31] D. Luitel, S. Hassani, and M. Sabetzadeh, "Improving requirements completeness: Automated assistance through large language models," *Requirements Engineering*, vol. 29, no. 1, pp. 73–95, 2024.
- [32] A. Ferrari, G. Gori, B. Rosadini, I. Trotta, S. Bacherini, A. Fantechi, and S. Gnesi, "Detecting requirements defects with nlp patterns: an industrial experience in the railway domain," *Empirical Software Engineering*, vol. 23, no. 6, pp. 3684–3733, 2018.
- [33] B. Gleich, O. Creighton, and L. Kof, "Ambiguity detection: Towards a tool explaining ambiguity sources," in *Requirements Engineering: Foundation for Software Quality: 16th International Working Conference, REFSQ 2010, Essen, Germany, June 30–July 2, 2010. Proceedings 16*. Springer, 2010, pp. 218–232.
- [34] M. Zakeri-Nasrabadi and S. Parsa, "Natural language requirements testability measurement based on requirement smells," *Neural Computing and Applications*, vol. 36, no. 21, pp. 13 051–13 085, 2024.
- [35] P. Szymański and T. Kajdanowicz, "A scikit-based Python environment for performing multi-label classification," *ArXiv e-prints*, Feb. 2017.
- [36] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [37] M. R. Costa-Jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard *et al.*, "No language left behind: Scaling human-centered machine translation," *arXiv preprint arXiv:2207.04672*, 2022.
- [38] S. Banerjee and A. Lavie, "Meteor: An automatic metric for mt evaluation with improved correlation with human judgments," in *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, 2005, pp. 65–72.
- [39] Google LLC, "Understanding the BLEU score," <https://cloud.google.com/translate/automl/docs/evaluate>, 2020.
- [40] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [41] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [42] S. Alharbi, "Ambiguity detection in requirements classification task using fine-tuned transformation technique," in *CS & IT Conference Proceedings*, vol. 12, no. 21. CS & IT Conference Proceedings, 2022.
- [43] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.
- [44] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [45] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.
- [46] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *biometrics*, pp. 159–174, 1977.
- [47] K. Kaur and P. Kaur, "Mnor-bert: multi-label classification of non-functional requirements using bert," *Neural Computing and Applications*, vol. 35, no. 30, pp. 22 487–22 509, 2023.
- [48] A. Berger and S. Guda, "Threshold optimization for f measure of macro-averaged precision and recall," *Pattern Recognition*, vol. 102, p. 107250, 2020.
- [49] A. Doknic and T. Möller, "Mlmc: Interactive multi-label multi-classifier evaluation without confusion matrices," *arXiv preprint arXiv:2501.14460*, 2025.
- [50] P. Dhal and C. Azad, "A multi-stage multi-objective gwo based feature selection approach for multi-label text classification," in *International Conference on Intelligent Technologies*. IEEE, 2022, pp. 1–5.
- [51] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv e-prints*, pp. arXiv–2407, 2024.
- [52] G. Team, M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.